TECHPULSE

Independent Academic and Technical Audit

Strict repository evaluation, scoring and actionable improvement report

Prepared from the checked-out implementation, tests, configuration, data artifacts and documentation. No application source was modified.

OVERALL SCORE **60 / 100**

Classification: Adequate but technically or academically limited

Contents

- 1. Executive summary
- 2. Scope and inspection method
- 3. Architecture and workflow audit
- 4. Scoring table and academic verdict
- 5. Category assessments
- 6. Streamlit UX review
- 7. API and live-intelligence review
- 8. Kenyan ecosystem relevance
- 9. Actionable improvement register
- 10. Four-phase roadmap and top ten
- 11. Appendix: evidence and verification limitations

1. Executive Summary

TechPulse is a credible academic prototype with a real Streamlit application, a modular Python pipeline, model comparison, explainability artifacts, tests, CI and a bounded live-observation subsystem. It is not yet a defensible technology-trajectory forecasting system. The central weakness is not presentation: it is the relationship between the target, historical observations, and the evaluation design.

- Overall audit score: 60/100, interpreted as Adequate but technically and academically limited.
- Strongest area: Streamlit and live-intelligence breadth. The app has nine navigable pages and source-specific collection code.
- Weakest area: machine-learning research validity. The committed model report gives 1.0 results for all candidates and the momentum rule while ROC-AUC is unavailable; this is a warning sign, not evidence of forecasting skill.
- The local Stack Exchange history spans 2016-01-01 to 2018-09-09, while survey-derived usage begins in 2021. The repository itself acknowledges the as-of mismatch.
- The live APIs are useful descriptive signals, but stars, questions, releases and downloads cannot be treated as adoption, employer demand or Kenyan usage.
- The project is suitable for a strong final-year project if the research question is narrowed, temporal validation is rebuilt, data lineage is made reproducible, and claims are constrained.

2. Scope and Inspection Method

The audit examined source code and executable configuration rather than awarding credit from documentation alone. The inspection followed the application path from source data to ingestion, feature creation, labels, model artifacts, dashboard loaders, live collection and persistence.

- Inspected entry points: streamlit_app.py and dashboard/app.py; all dashboard views and reusable components.
- Inspected data and ML code: src/data_ingestion.py, feature_engineering.py, labelling.py, technology_normalization.py, model_training.py, evaluation.py, explain.py, shap_analysis.py and pipeline/run_pipeline.py.
- Inspected live collection and storage: src/live_data/collectors.py, service.py, storage.py, scripts/collect_live_data.py and schedule_live_data.py.
- Inspected requirements, Dockerfile, pytest.ini, CI workflows, SQL queries, provenance, methodology, user and technical documentation.
- Executed pytest -q: 49 passed, 6 warnings. Executed compileall successfully. flake8 produced many style violations in dashboard/views. A dashboard AppTest startup check was attempted; no source files were changed.
- No credentials were read or printed. No application source, configuration, data, model or documentation files were modified.

3. Architecture and End-to-End Workflow

The deployed path is dashboard/app.py, which registers nine st.Page routes. Dashboard loaders in dashboard/data.py read cached CSV and JSON artifacts. The research pipeline is separately orchestrated by pipeline/run_pipeline.py and uses src/ modules. Live collection is deliberately additive: it writes observations to PostgreSQL when configured or to outputs/live_data/live_observations.csv, and the live page does not directly train the historical model.

- Source layer: local CSVs include survey aggregates, a 2016-2018 Stack Exchange export, Fortune 500 metadata and an empty data/files.csv.
- Transformation layer: ingestion normalizes source tables; feature engineering builds Stack Exchange, survey, enterprise-proxy and sentiment-derived signals; labels are derived from future activity.
- Model layer: Logistic Regression, KNN, Random Forest and XGBoost are compared with majority and momentum baselines; imputation and model artifacts are serialized.
- Presentation layer: cached artifact readers, rankings, explorer, model performance, methodology and live intelligence pages.
- Architectural concern: streamlit_app.py is a legacy/parallel dashboard implementation while Docker and documented commands use dashboard/app.py. This increases maintenance and test ambiguity.
- Positive boundary: live observations are not silently merged into historical training data; this is methodologically correct, but it means live data currently strengthens descriptive intelligence rather than forecasting.

4. Scoring Table

Category	Maximum	Awarded	Rationale
Research problem and originality	10	7	Relevant and useful framing, but novelty is mainly integration of public signals; causal or predictive contribution is not established.
Data engineering and quality	15	9	Good provenance and defensive ingestion, but historical coverage, proxy data and temporal alignment are limiting.
ML and research methodology	15	7	Multiple candidates and baselines exist, but perfect results, missing ROC-AUC and weak temporal evidence make claims fragile.
PostgreSQL and database engineering	10	6	SQLAlchemy/PostgreSQL path and additive table exist; schema breadth, migrations and real PostgreSQL testing are limited.
Streamlit UX	15	11	Polished multi-page structure, caching and explanatory pages; usability, accessibility and interaction depth remain uneven.
APIs and live intelligence	10	8	Four realistic collectors with rate handling and metadata; bounded samples and descriptive-only semantics limit research value.
Software engineering and reliability	10	8	Modular code, CI, Docker and defensive errors; duplicate entry point and lint debt reduce maintainability.
Testing, documentation and reproducibility	5	4	Strong tests and substantial docs; no measured coverage target, pinned lockfile or PostgreSQL integration environment.
TOTAL	100	60	Adequate but technically or academically limited

5. Academic Verdict

Verdict: Adequate but technically or academically limited. The repository demonstrates substantial engineering effort and unusually good candour about limitations. An examiner can credit the modular implementation, reproducibility intent, baselines, provenance, test suite and live-data separation. An examiner can also challenge whether the system predicts technology trajectories at all.

- Genuinely impressive: end-to-end breadth; explicit source provenance; defensive collectors; additive live storage; model baselines; SHAP artifacts; CI and Docker packaging; 49 passing tests.
- Academically weak: labels appear to be a discretization of future observed activity rather than an externally validated trajectory construct; one small holdout can produce misleadingly perfect metrics.
- Claims difficult to defend: “prediction” as future technology adoption; enterprise adoption inferred from Fortune metadata/proxies; current API counts as historical evidence; global signals as Kenyan ecosystem evidence.
- Grade risk: if the examiner inspects the data period, model report and target construction, 1.0 scores for four models plus a momentum rule will look more like a separability or leakage problem than a robust result.
- The best academic repositioning is a transparent research aid for comparative technology momentum, with forecasting presented as exploratory until temporal backtesting and external labels exist.

6. Data Engineering and Quality Audit

Evidence supports a reasonably organized ingestion boundary, but not a strong empirical basis for broad technology-adoption claims.

- Credible source: Stack Overflow Developer Survey data is documented with license, URLs, transformations and 2021-2024 derived aggregates in outputs/data_provenance.json.
- Historical community data: data/stackexchange.csv has 45,238 rows and dates only from 2016 to 2018. It is not contemporaneous with the survey data and is not a continuous modern panel.
- Development proxies: the repository and README acknowledge deterministic development proxies for company adoption, metadata and mappings. These must never be described as measured enterprise use.
- Schema weakness: data/files.csv is empty and several source contracts are inferred by flexible column aliases; a formal schema registry and row-level validation report are missing.
- Reproducibility gap: provenance records downloaded raw survey files, but the committed lightweight dataset is derived and the full raw files are not visible in the repository inventory.
- Required correction: publish a versioned data manifest with source hash, row counts, date bounds, missingness, duplicate counts, transformation version and as-of availability for every feature.

7. Machine Learning and Methodology Audit

This is the most important correction area. src/model_training.py includes candidate models, imputation and cross-validation logic; src/evaluation.py evaluates held-out artifacts and adds majority and momentum baselines. Those are good components, but the current evidence does not establish generalizable forecasting.

- Committed outputs/model_comparison.md reports xgboost, logistic regression, KNN, random forest and baseline_momentum_rule at 1.0 accuracy and weighted F1; majority baseline is 0.667 accuracy and 0.533 weighted F1.
- Macro ROC-AUC is NaN because the held-out split lacks all classes. Reporting perfect accuracy without a valid multiclass discrimination metric is not sufficient.
- A single or tiny holdout is sensitive to technology-level leakage and class construction. The momentum rule matching every candidate is a strong diagnostic signal.
- A defensible design needs expanding-window or rolling-origin validation, a strict technology-and-time grouping policy, per-class support, confidence intervals, calibration, and a naive last-period baseline.
- The target needs an explicit operational definition: forecast horizon, minimum observation threshold, treatment of new technologies, and whether the label is a change in volume, share or rank.
- SHAP improves explanation of fitted behavior but does not prove causal importance, stability or out-of-time usefulness.

8. PostgreSQL and Storage Audit

PostgreSQL is more than a README placeholder: src/data_ingestion.py supports a configured database boundary and src/live_data/storage.py creates an additive live-observation table through SQLAlchemy. However, the production database contract is incomplete.

- Strengths: environment-based DATABASE_URL, SQLAlchemy persistence, CSV fallback, natural-key deduplication for live observations and separate live storage semantics.
- Verification limit: the tests exercise live persistence with temporary SQLite, not a PostgreSQL service. PostgreSQL-specific types, indexes, permissions and transaction behavior are therefore unverified.
- The 12 SQL analytics queries appear to be an educational query set, not a complete normalized schema/migration system. No Alembic migration history or database bootstrap script was found.
- The CSV fallback and PostgreSQL paths need contract tests that compare schema, null handling, sort order, deduplication and timestamp semantics.
- Recommended design: migrations for source tables and live observations, foreign keys to a technology dimension, CHECK constraints for metric types, indexes on technology/source/metric/observed_at, and a database health page.

9. Streamlit UX Audit

The deployed dashboard has a coherent information architecture: Start Here, Technology Explorer, Rankings, Community Health, Prediction Explorer, Model Performance & Validation, Methodology & Data, Live Technology Intelligence, and About & User Guide. This is a strong foundation. The application still presents several advanced outputs before establishing a clear mental model for non-specialists.

- Good: wide layout, material navigation icons, cached artifact reads, defensive empty-file handling, separate methodology and data pages, and explicit limitations in the live-data design.
- Improve onboarding: Start Here should show one primary task, a compact data freshness panel, “what this score means”, and a worked technology example linked to the explorer.
- Improve exploration: add a single technology search route with comparable global signals, date range, source badges, missing-data indicators and links to the underlying observations.
- Improve predictions: display class support, forecast horizon, training cutoff, calibration/reliability, baseline comparison and an uncertainty statement next to every prediction.
- Improve operational states: consistent loading, stale-data, API failure, no-match and partial-source states; current defensive loaders can silently return empty frames, which needs visible provenance.
- Accessibility and performance: audit color-only statuses, keyboard navigation, chart labels, mobile layout, large tables and repeated SHAP computation in the prediction form.
- Style debt: flake8 reports extensive E225/E231/E701 violations in views including Prediction Explorer and Methodology Data. This is not cosmetic only; dense one-line code is harder to audit.

10. API and Live Intelligence Review

The live subsystem is the most realistic advanced feature in the repository. It uses bounded, source-specific collectors and stores timestamped observations. The correct interpretation is “public ecosystem activity signals”, not technology adoption.

Source	Useful contribution	Limitations and role	Priority
GitHub REST	Repository counts, stars, forks,	Search-result bias, sampled repositories,	High
Stack Exchange	Tag volume, answered/unanswered activity, recent	One site, capped pagination, community	High
PyPI JSON	Version and release history for mapped	Package release metadata, no trustworthy	Medium
npm Registry	Release dates and recent download	Downloads are package events, not unique users	Medium
deps.dev	Dependency graph and advisories	Potentially valuable health/maintenance context; not	Medium
Kenyan sources	Employer demand and local adoption if	Requires local job postings, surveys,	Critical for local claims

11. Kenyan Developer Ecosystem Relevance

TechPulse can become useful in Kenya, but its current data cannot support claims about Kenyan developers or organisations. Global popularity, global enterprise adoption, Kenyan employer demand and Kenyan organisational use are distinct variables.

- Students and junior developers: a transparent comparison tool could show learning relevance, community support and recency, provided labels are not framed as career guarantees.
- Startups and professionals: ecosystem health and release/dependency signals may inform due diligence, but they do not establish local hiring demand or local support capacity.
- Required local evidence: periodically sampled Kenyan job adverts with location/employer/date, structured interviews or survey responses from Kenyan developers, local training-course curricula, and optionally anonymized organisation technology surveys.
- Method: publish sampling frame, deduplicate adverts, distinguish remote/global roles from Kenya-based roles, code technology mentions consistently, and report uncertainty and missingness.
- Do not merge Kenyan labour-market labels into the global model until there is enough temporal coverage and a documented representativeness argument. Start with a separate Kenyan descriptive module.

12. Actionable Improvement Register: Critical

Each item below is intended to be handed directly to an implementer. Effort estimates assume one developer familiar with the repository.

- TP-01 | Rebuild evaluation around time. Problem: current perfect scores and missing ROC-AUC do not demonstrate forecasting. Evidence: outputs/model_comparison.md; src/evaluation.py. Solution: create rolling-origin backtests by technology-month, freeze features at cutoff, report macro-F1, balanced accuracy, per-class support, calibration and confidence intervals. Location: src/model_training.py, src/evaluation.py, tests/. Outcome: defensible out-of-time estimate. Priority Critical; Difficult; 5-8 days.

- TP-02 | Audit target construction and leakage. Problem: label semantics and feature/label horizon are not sufficiently examiner-proof. Evidence: src/feature_engineering.py and src/labelling.py. Solution: write a target specification, enforce future-only columns, assert cutoff dates, and add adversarial shuffled-time tests. Priority Critical; Difficult; 3-5 days.

- TP-03 | Separate proxies from observations. Problem: deterministic enterprise proxies can be mistaken for measured adoption. Evidence: outputs/data_provenance.json, README.md, feature engineering. Solution: typed source registry with observed/proxy/synthetic flags propagated into feature and UI metadata; exclude proxies from headline claims. Priority Critical; Moderate; 2-4 days.

- TP-04 | Make data reproducibility measurable. Problem: raw acquisition and derived data cannot be reconstructed from a single manifest. Solution: add acquisition scripts with hashes, schema snapshots, row/missingness/duplicate reports and a run manifest containing code version and parameters. Priority Critical; Moderate; 3-5 days.

- TP-05 | Add real PostgreSQL integration. Problem: SQLite tests do not verify PostgreSQL. Solution: CI service container, migration/bootstrap SQL, isolated test schema, rollback tests and contract comparison with CSV fallback. Location: src/data_ingestion.py, src/live_data/storage.py, .github/workflows. Priority Critical; Difficult; 4-7 days.

13. Actionable Improvement Register: Major

- TP-06 | Consolidate entry points. Problem: streamlit_app.py and dashboard/app.py create two application paths. Solution: retire or explicitly label the legacy path, make one launcher the source of truth, and test the Docker command. Priority High; Easy-Moderate; 1-2 days.

- TP-07 | Add model reliability UI. Problem: users see a class/confidence without enough context. Solution: show training cutoff, horizon, class prevalence, baseline, calibration plot and “not a decision” language beside predictions. Location: dashboard/views/3_Model_Performance.py and 6_Prediction_Explorer.py. Priority High; Moderate; 2-4 days.

- TP-08 | Build a technology detail workflow. Problem: pages provide views but not a unified evidence trail. Solution: search/select one technology, show source cards, date coverage, trend chart, model result, SHAP explanation and missing-source warnings. Location: dashboard/views/1_Home.py, components/tech_detail_panel.py, trend_charts.py. Priority High; Moderate; 3-5 days.

- TP-09 | Add freshness and provenance contracts. Problem: empty/corrupt artifacts can become silent empty states. Solution: central artifact manifest, visible last-updated/source/type badges, error severity and stale thresholds. Location: dashboard/data.py, 7_Methodology_Data.py. Priority High; Moderate; 2-3 days.

- TP-10 | Establish coverage and style gates. Problem: 49 tests pass but coverage and lint quality are not governed. Solution: pytest-cov threshold by module, flake8/formatter gate, dashboard AppTest assertions for every route and no warnings/errors. Priority High; Moderate; 2-3 days.

- TP-11 | Use migrations and indexed schema. Problem: first-write table creation is not a deployment strategy. Solution: Alembic or versioned SQL migrations; indexes and constraints; technology dimension and source dimension. Priority High; Difficult; 4-6 days.

- TP-12 | Formalize live API semantics. Problem: current signals can be overread. Solution: metric dictionary documenting unit, population, collection window, rate limit, missingness and whether a metric is eligible for ML. Priority High; Moderate; 2-3 days.

14. Actionable Improvement Register: Advanced and Polish

- TP-13 | Add an evidence-quality score. Combine recency, coverage, source type, sample size and collection health; use it to qualify rankings. Priority Medium; Moderate; 3-4 days.
- TP-14 | Add deps.dev only for dependency health. Use advisories, dependency counts and release freshness as a separate engineering-health view, not an adoption label. Priority Medium; Moderate; 3-5 days.
- TP-15 | Build a Kenyan module separately. Acquire ethically collected job/survey evidence, document representativeness, and present descriptive findings before any local model. Priority Medium; Difficult; 10-15 days plus data collection.
- TP-16 | Add calibration and uncertainty. Use cross-validated calibration, bootstrap intervals and abstention for low-support technologies. Priority High; Difficult; 4-7 days.
- TP-17 | Improve dashboard accessibility. Replace color-only status, add alt text/captions, check keyboard flow, responsive tables and chart labels. Priority Medium; Moderate; 2-4 days.
- TP-18 | Package an examiner evidence bundle. Include one clean reproduction command, run manifest, data dictionary, model card, limitations and a short demo dataset. Priority Medium; Easy; 2-3 days.

15. Streamlit Redesign Recommendations

Existing surface	Specific change	Why it helps
0_Start_Here.py	Add three-step onboarding and a sample technology CTA	Makes purpose and first action obvious
1_Home.py + tech_detail_panel.py	Unified search/detail view with source/date coverage	Turns separate charts into an evidence trail
2_Rankings.py	Add evidence-quality, as-of date, sample size and baseline columns	Prevents false precision in rankings
3_Model_Performance.py	Show per-class support, calibration, confidence intervals and baseline delta	Makes validation interpretable to beginners and examiners
5_Community_Health.py	Separate community activity from adoption; show window and coverage	Prevents Stack Exchange counts being overread
6_Prediction_Explorer.py	Add horizon, cutoff, class prevalence and abstain state	Makes predictions safer and more honest
7_Methodology_Data.py	Interactive source registry and downloadable run manifest	Improves reproducibility and trust
8_Live_Technology_Intelligence.py	Source health, last successful collection, stale warning and metric dictionary	Makes API failure and irregular collection visible
All pages	Consistent loading, empty, stale and partial-source states; no color-only meaning	Improves accessibility and operational clarity

16. Four-Phase Roadmap

Phase	Order and work	Dependencies	Impact
1. Critical corrections	TP-01, TP-02, TP-03, TP-04, TP-05	Target specification; database service;	Transforms the project from plausible prototype to defensible study
2. Major enhancements	TP-06 through TP-12; dashboard	Phase 1 artifacts and metadata contracts	Improves maintainability, trust and repeated use
3. Advanced intelligence	TP-13 through TP-17; validated	Historical collection, source semantics,	Adds useful intelligence without overclaiming
4. Academic/professional polish	TP-18, final model card, user manual, deployment	Stable results and documented limitations	Improves examiner confidence and demonstration quality

17. Top Ten Highest-Impact Changes

- 1. Replace the single holdout with rolling-origin, out-of-time evaluation.
- 2. Publish and enforce a precise target and leakage specification.
- 3. Remove or visibly quarantine synthetic/development proxies from headline claims.
- 4. Add a versioned acquisition manifest with hashes, schema and quality reports.
- 5. Test the actual PostgreSQL path in CI and introduce migrations.
- 6. Consolidate the two Streamlit entry points.
- 7. Add calibration, uncertainty, support and baseline context to prediction UI.
- 8. Create one technology detail page that joins all sources with as-of dates.
- 9. Add freshness, provenance and partial-failure states across dashboard pages.
- 10. Add coverage/lint gates and an examiner-ready reproducibility bundle.

18. Appendix: Evidence Register

- Implementation: dashboard/app.py, dashboard/data.py, dashboard/views/*.py, dashboard/components/*.py.
- Data/ML: src/data_ingestion.py, src/feature_engineering.py, src/labelling.py, src/model_training.py, src/evaluation.py, src/explain.py, src/shap_analysis.py, pipeline/run_pipeline.py.
- Live/API: src/live_data/collectors.py, src/live_data/service.py, src/live_data/storage.py, scripts/collect_live_data.py, scripts/schedule_live_data.py, data/live_package_map.json.
- Verification: tests/*.py, pytest.ini, .github/workflows/ci.yml, .github/workflows/ml-pipeline.yml, Dockerfile, requirements.txt.
- Evidence artifacts: outputs/model_comparison.md, outputs/model_comparison.csv, outputs/data_provenance.json, outputs/live_data_test_report.md, docs/methodology.md, docs/LIVE_DATA_IMPLEMENTATION_PLAN.md.
- Measured facts used: pytest 49 passed with 6 warnings; survey aggregate 685 rows/267 technologies/2021-2024; Stack Exchange CSV 45,238 rows/2016-2018; Fortune CSV 500 rows; empty data/files.csv; model report values recorded above; flake8 style violations in dashboard views.

19. Verification Limitations and Final Conclusion

This report is an independent repository audit, not a claim that every external API or deployment environment was exercised. The environment did not contain a PostgreSQL service, so database behavior was verified through code inspection and SQLite-backed tests rather than a live PostgreSQL integration. External API collection was not repeated during this audit. Visual judgment was based on source, existing evidence artifacts and dashboard smoke tooling rather than a full manual browser study of every viewport.

- No credentials or private environment values were inspected.
- No application source was modified; only the requested PDF was generated.
- The project should not present the current model numbers as evidence of real-world adoption forecasting.
- The fastest route to excellence is methodological: fix time, target, leakage and data provenance first; then refine the interface around trustworthy evidence.
- Final score: 60/100. Category interpretation: Adequate but technically or academically limited.